

Chapter 52 — Probability and Randomized Algorithms

0. What You Will Learn in This Chapter

Probability and randomized algorithms appear in interviews when the problem involves:

- random selection
- weighted random choice
- shuffling
- sampling from a stream
- expected value
- probability reasoning
- designing fair random generators
- picking random points
- randomized data structures
- avoiding bias
- uniform distribution

This chapter is especially useful for DE Shaw-style interviews because probability reasoning is commonly tested in trading, quant, and algorithm-heavy environments. By the end of this chapter, you should understand:

- Basic probability
- Conditional probability
- Independence
- Expected value
- Linearity of expectation
- Uniform random selection
- Weighted random selection
- Prefix sums with binary search for randomness
- Reservoir sampling
- Fisher-Yates shuffle
- Randomized Set
- Random Pick Index
- Random Pick With Weight
- Random Point in Circle
- Random Point in Rectangles
- Rejection sampling
- How to avoid modulo/random bias
- Monte Carlo vs Las Vegas algorithms
- Common probability interview traps

Core problems covered in this chapter:

1. Random Pick Index
2. Random Pick With Weight
3. Insert Delete GetRandom O(1)
4. Shuffle an Array
5. Linked List Random Node
6. Random Point in Non-overlapping Rectangles
7. Generate Random Point in a Circle
8. Implement Rand10 Using Rand7

9. Random Pick With Blacklist
 10. Random Flip Matrix
 11. Randomized Collection with Duplicates
 12. Reservoir Sampling
 13. Fisher-Yates Shuffle
 14. Expected Value Basics
 15. Probability Linearity Examples
 16. Monte Carlo vs Las Vegas preview
-

1. What Is Probability?

Probability measures how likely an event is.

If all outcomes are equally likely:

probability = favorable outcomes / total outcomes

Example:

A fair six-sided die.

Probability of rolling 4:

1 / 6

Probability of rolling an even number:

3 / 6 = 1 / 2

because even outcomes are:

2, 4, 6

2. Why Probability Matters in Algorithms

Many algorithm problems require random behavior but with strict fairness.

Example:

Pick an index of target uniformly at random.

If target appears at indices:

[2, 5, 9]

each index must be returned with probability:

1 / 3

Another example:

Shuffle an array uniformly.

Every permutation must be equally likely.

For array of length n, there are:

n!

permutations.

A correct shuffle gives each permutation probability:

$1 / n!$

3. Probability Vocabulary

3.1 Experiment

A process with uncertain outcome.

Example:

roll a die
flip a coin
pick random index

3.2 Sample Space

Set of all possible outcomes.

For die:

$\{1,2,3,4,5,6\}$

3.3 Event

A subset of outcomes.

Example:

roll even = $\{2,4,6\}$

3.4 Uniform Distribution

All outcomes equally likely.

Example:

A fair die has uniform distribution over 6 outcomes.

3.5 Weighted Distribution

Outcomes have different probabilities.

Example:

Weights:

$[1, 3]$

Index 0 should be picked with probability:

$1 / 4$

Index 1 should be picked with probability:

$3 / 4$

4. Basic Probability Rules

4.1 Complement Rule

$P(\text{not } A) = 1 - P(A)$

Example:

Probability of not rolling 6:

$$1 - 1/6 = 5/6$$

4.2 Addition Rule

If A and B cannot both happen:

$$P(A \text{ or } B) = P(A) + P(B)$$

Example:

Rolling 1 or 2:

$$1/6 + 1/6 = 2/6$$

If they can overlap:

$$P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$$

4.3 Multiplication Rule

If A and B are independent:

$$P(A \text{ and } B) = P(A) * P(B)$$

Example:

Flip two coins.

Probability both heads:

$$1/2 * 1/2 = 1/4$$

5. Conditional Probability

Conditional probability means probability of A given B happened.

$$P(A | B) = P(A \text{ and } B) / P(B)$$

Example:

Roll a die.

Given the number is even, probability it is 4:

Even outcomes:

2, 4, 6

Favorable:

4

Probability:

1 / 3

6. Independence

Two events are independent if one does not affect the other.

Example:

Two coin flips are independent.

But drawing cards without replacement is not independent.

If first card is an ace, probability second card is an ace changes.

In algorithms, independence matters when generating random values.

7. Expected Value

Expected value is the average outcome over many trials.
For a random variable X:

$$E[X] = \text{sum}(\text{value} * \text{probability})$$

Example:

Fair die expected value:

$$\begin{aligned} &1*(1/6) + 2*(1/6) + 3*(1/6) + 4*(1/6) + 5*(1/6) + 6*(1/6) \\ &= 21/6 \\ &= 3.5 \end{aligned}$$

Expected value does not have to be an outcome. You cannot roll 3.5, but average approaches 3.5.

8. Linearity of Expectation

This is one of the most important probability tools.

For random variables X and Y:

$$E[X + Y] = E[X] + E[Y]$$

This is true even if X and Y are not independent.

This is powerful for counting expected number of events.

Example:

Expected number of heads in 10 fair coin flips:

Let $X_i = 1$ if flip i is heads, else 0.

$$E[X_i] = 1/2$$

Total heads:

$$X = X_1 + X_2 + \dots + X_{10}$$

So:

$$E[X] = 10 * 1/2 = 5$$

9. Randomized Algorithm Types

9.1 Las Vegas Algorithm

Always gives correct answer, but runtime may vary.

Example:

Randomized quicksort.

It always sorts correctly, but pivot choices affect time.

9.2 Monte Carlo Algorithm

Runtime is fixed or bounded, but answer may be wrong with small probability.

Example:

Probabilistic primality testing.

9.3 Interview Rule

Most LeetCode random design problems expect exact probability distribution, not approximate Monte Carlo.

10. C++ Random Tools

For interview simplicity, people often use:

```
rand()
```

But modern C++ prefers:

```
mt19937 rng(random_device{}());  
uniform_int_distribution<int> dist(l, r);
```

Example:

```
class RandomHelper {  
public:  
    mt19937 rng;  
  
    RandomHelper() {  
        rng = mt19937(random_device{}());  
    }  
  
    int randomInt(int left, int right) {  
        uniform_int_distribution<int> dist(left, right);  
        return dist(rng);  
    }  
};
```

For LeetCode, `rand()` is usually accepted, but be aware of modulo bias.

11. Modulo Bias

A common mistake:

```
rand() % n
```

This may be biased if `RAND_MAX + 1` is not divisible by `n`.

Example:

If random generator gives values 0 to 9 uniformly and you do `% 6`:

```
0,6 -> 0  
1,7 -> 1  
2,8 -> 2  
3,9 -> 3  
4  -> 4  
5  -> 5
```

0,1,2,3 occur twice, while 4,5 occur once.

Biased.

In interviews, many solutions still use %, but for correctness discussions, mention rejection sampling or `uniform_int_distribution`.

12. Core Template 1 — Uniform Random Integer

Using modern C++:

```
class RandomInt {
public:
    mt19937 rng;

    RandomInt() {
        rng = mt19937(random_device{}());
    }

    int get(int left, int right) {
        uniform_int_distribution<int> dist(left, right);
        return dist(rng);
    }
};
```

13. Core Template 2 — Weighted Random Pick

If weights are:

[2, 5, 3]

Build prefix sums:

[2, 7, 10]

Total = 10.

Pick random number x from:

1 to 10

Then:

x in 1..2 -> index 0
x in 3..7 -> index 1
x in 8..10 -> index 2

Use binary search.

```
int pickIndex() {
    int x = random number from 1 to total;
    return lower_bound(prefix.begin(), prefix.end(), x) - prefix.begin();
}
```

14. Core Template 3 — Reservoir Sampling

Used when:

- stream length is unknown
- cannot store all items
- need uniformly random item

For one item:

At item count k , replace current answer with probability:

$1 / k$

Template:

```
int answer;
int count = 0;

for each item x:
    count++;

    if (randomInt(1, count) == 1) {
        answer = x;
    }
```

At the end, each item has probability:

$1 / n$

15. Core Template 4 — Fisher-Yates Shuffle

Correct uniform shuffle.

For i from $n - 1$ down to 1 :

```
pick j randomly from  $[0, i]$ 
swap  $a[i], a[j]$ 
for (int i = n - 1; i > 0; i--) {
    int j = randomInt(0, i);
    swap(nums[i], nums[j]);
}
```

This gives every permutation equal probability.

16. Full Problem Walkthrough 1 — LeetCode 398: Random Pick Index

16.1 Problem Summary

Given array nums.

Implement:

```
pick(target)
```

Return a random index i such that:

```
nums[i] == target
```

All valid indices must be equally likely.

16.2 Approach 1 — Hash Map of Indices

Preprocess all indices for each value.

Then pick random index from vector.

16.3 C++ Code

```
class Solution {
private:
    unordered_map<int, vector<int>> positions;
    mt19937 rng;

public:
    Solution(vector<int>& nums) {
        rng = mt19937(random_device{}());

        for (int i = 0; i < nums.size(); i++) {
            positions[nums[i]].push_back(i);
        }
    }

    int pick(int target) {
        vector<int>& arr = positions[target];

        uniform_int_distribution<int> dist(0, arr.size() - 1);

        return arr[dist(rng)];
    }
};
```

16.4 Complexity

Constructor:

$O(n)$

Pick:

$O(1)$

Space:

$O(n)$

17. Random Pick Index With Reservoir Sampling

17.1 Why Use Reservoir Sampling?

If you do not want extra $O(n)$ memory, scan nums each time.
For every occurrence of target, choose it with probability:

$1 / \text{count}$

17.2 C++ Code

```
class Solution {
private:
    vector<int> nums;
    mt19937 rng;

public:
    Solution(vector<int>& nums) {
        this->nums = nums;
        rng = mt19937(random_device{}());
    }

    int pick(int target) {
        int answer = -1;
        int count = 0;

        for (int i = 0; i < nums.size(); i++) {
            if (nums[i] == target) {
                count++;

                uniform_int_distribution<int> dist(1, count);

                if (dist(rng) == 1) {
                    answer = i;
                }
            }
        }
    }
}
```

```

    }

    return answer;
}
};

```

17.3 Why It Is Uniform

Suppose target appears k times.

For occurrence i to be final answer:

- It must be chosen when seen: probability $1/i$.
- It must not be replaced by later occurrences $i+1$ to k .

Probability not replaced at step j :

$$1 - 1/j = (j - 1) / j$$

So final probability:

$$1/i * i/(i+1) * (i+1)/(i+2) * ... * (k-1)/k = 1/k$$

Every occurrence has probability $1/k$.

18. Full Problem Walkthrough 2 — LeetCode 528: Random Pick With Weight

18.1 Problem Summary

Given weights w .

Pick index i with probability:

$$w[i] / \text{sum}(w)$$

Example:

$$w = [1, 3]$$

Index 0 probability:

$$1 / 4$$

Index 1 probability:

$$3 / 4$$

18.2 Prefix Sum Idea

Weights represent intervals.

For:

$$[1, 3]$$

Prefix:

$$[1, 4]$$

Random x in $1..4$:

x = 1 -> index 0
x = 2,3,4 -> index 1

18.3 C++ Code

```
class Solution {
private:
    vector<int> prefix;
    int total;
    mt19937 rng;

public:
    Solution(vector<int>& w) {
        rng = mt19937(random_device{}());

        total = 0;

        for (int x : w) {
            total += x;
            prefix.push_back(total);
        }
    }

    int pickIndex() {
        uniform_int_distribution<int> dist(1, total);

        int x = dist(rng);

        return lower_bound(prefix.begin(), prefix.end(), x) - prefix.begin();
    }
};
```

18.4 Complexity

Constructor:

O(n)

Pick:

O(log n)

Space:

O(n)

19. Full Problem Walkthrough 3 — LeetCode 380: Insert Delete GetRandom O(1)

19.1 Problem Summary

Design a data structure supporting:

```
insert(val)
remove(val)
getRandom()
```

all average $O(1)$.

19.2 Key Idea

Use:

- vector to store values
- hash map from value to index

For random:

```
pick random vector index
```

For remove:

- swap element with last element
- update moved element index
- pop back

19.3 C++ Code

```
class RandomizedSet {
private:
    vector<int> values;
    unordered_map<int, int> index;
    mt19937 rng;

public:
    RandomizedSet() {
        rng = mt19937(random_device{}());
    }

    bool insert(int val) {
        if (index.count(val)) {
            return false;
        }

        index[val] = values.size();
        values.push_back(val);
    }
};
```

```

        return true;
    }

    bool remove(int val) {
        if (!index.count(val)) {
            return false;
        }

        int idx = index[val];
        int lastVal = values.back();

        values[idx] = lastVal;
        index[lastVal] = idx;

        values.pop_back();
        index.erase(val);

        return true;
    }

    int getRandom() {
        uniform_int_distribution<int> dist(0, values.size() - 1);

        return values[dist(rng)];
    }
};

```

19.4 Important Edge Case

If removing the last element, the swap still works:

```
lastVal == val
```

Then erase val afterward.

20. Full Problem Walkthrough 4 — LeetCode 384: Shuffle an Array

20.1 Problem Summary

Implement:

```
reset()
shuffle()
```

Shuffle must return uniformly random permutation.

20.2 Fisher-Yates Shuffle

For each position from end to beginning, choose random element from remaining prefix.

20.3 C++ Code

```
class Solution {
private:
    vector<int> original;
    mt19937 rng;

public:
    Solution(vector<int>& nums) {
        original = nums;
        rng = mt19937(random_device{}());
    }

    vector<int> reset() {
        return original;
    }

    vector<int> shuffle() {
        vector<int> arr = original;

        for (int i = arr.size() - 1; i > 0; i--) {
            uniform_int_distribution<int> dist(0, i);
            int j = dist(rng);

            swap(arr[i], arr[j]);
        }

        return arr;
    }
};
```

20.4 Why Fisher-Yates Is Uniform

For each final position i , every remaining element has equal chance to be placed there.

Probability of a particular permutation:

$$1/n * 1/(n-1) * \dots * 1/1 = 1/n!$$

So every permutation is equally likely.

21. Full Problem Walkthrough 5 — LeetCode 382: Linked List Random Node

21.1 Problem Summary

Given head of singly linked list.
Return random node value.
Each node must be equally likely.
Length may be unknown.

21.2 Reservoir Sampling

As we traverse:

- first node selected with probability 1
- second replaces with probability $1/2$
- kth replaces with probability $1/k$

21.3 C++ Code

```
class Solution {
private:
    ListNode* head;
    mt19937 rng;

public:
    Solution(ListNode* head) {
        this->head = head;
        rng = mt19937(random_device{}());
    }

    int getRandom() {
        int answer = head->val;
        int count = 0;

        ListNode* node = head;

        while (node) {
            count++;

            uniform_int_distribution<int> dist(1, count);

            if (dist(rng) == 1) {
                answer = node->val;
            }

            node = node->next;
        }

        return answer;
    }
};
```

```
        node = node->next;
    }

    return answer;
}
};
```

21.4 Complexity

Per call:

$O(n)$

Space:

$O(1)$

22. Full Problem Walkthrough 6 — LeetCode 497: Random Point in Non-overlapping Rectangles

22.1 Problem Summary

Given non-overlapping axis-aligned rectangles with integer coordinates.

Pick a random integer point uniformly from all points covered by rectangles.

Important:

A rectangle:

$[x1, y1, x2, y2]$

contains:

$(x2 - x1 + 1) * (y2 - y1 + 1)$

integer points.

22.2 Pattern

Weighted random rectangle selection.

Weight = number of integer points in rectangle.

Then pick random point inside selected rectangle.

22.3 C++ Code

```
class Solution {
private:
    vector<vector<int>>> rects;
    vector<int> prefix;
    int total;
    mt19937 rng;

public:
```

```

Solution(vector<vector<int>>& rects) {
    this->rects = rects;
    rng = mt19937(random_device{}());

    total = 0;

    for (auto& r : rects) {
        int points = (r[2] - r[0] + 1) * (r[3] - r[1] + 1);
        total += points;
        prefix.push_back(total);
    }
}

vector<int> pick() {
    uniform_int_distribution<int> distPoint(1, total);
    int x = distPoint(rng);

    int idx = lower_bound(prefix.begin(), prefix.end(), x) - prefix.begin();

    auto& r = rects[idx];

    uniform_int_distribution<int> distX(r[0], r[2]);
    uniform_int_distribution<int> distY(r[1], r[3]);

    return {distX(rng), distY(rng)};
}
};

```

22.4 Why This Is Uniform

Rectangle is chosen with probability:

$\text{pointsInRectangle} / \text{totalPoints}$

Then each point inside is chosen with probability:

$1 / \text{pointsInRectangle}$

So each point overall:

$\text{pointsInRectangle} / \text{totalPoints} * 1 / \text{pointsInRectangle} = 1 / \text{totalPoints}$

23. Full Problem Walkthrough 7 — LeetCode 478: Generate Random Point in a Circle

23.1 Problem Summary

Given circle radius and center.

Generate random point uniformly inside circle.

23.2 Wrong Approach

Wrong:

```
r = random(0, radius)
theta = random(0, 2π)
```

Why wrong?

Area grows with r .

If radius is chosen uniformly, too many points cluster near center.

23.3 Correct Polar Approach

Area within radius r is proportional to:

r^2

So choose:

```
r = radius * sqrt(U)
```

where U is uniform in $[0, 1]$.

Choose angle:

```
theta = uniform(0, 2π)
```

23.4 C++ Code

```
class Solution {
private:
    double radius;
    double xCenter;
    double yCenter;
    mt19937 rng;
    uniform_real_distribution<double> dist;

public:
    Solution(double radius, double x_center, double y_center) {
        this->radius = radius;
        this->xCenter = x_center;
        this->yCenter = y_center;

        rng = mt19937(random_device{}());
```

```

        dist = uniform_real_distribution<double>(0.0, 1.0);
    }

    vector<double> randPoint() {
        double u = dist(rng);
        double theta = 2.0 * acos(-1.0) * dist(rng);

        double r = radius * sqrt(u);

        double x = xCenter + r * cos(theta);
        double y = yCenter + r * sin(theta);

        return {x, y};
    }
};

```

23.5 Alternative — Rejection Sampling

Pick x and y uniformly in bounding square.
Reject if outside circle.

```

vector<double> randPoint() {
    while (true) {
        double x = xCenter - radius + 2 * radius * dist(rng);
        double y = yCenter - radius + 2 * radius * dist(rng);

        double dx = x - xCenter;
        double dy = y - yCenter;

        if (dx * dx + dy * dy <= radius * radius) {
            return {x, y};
        }
    }
}

```

Expected attempts:

square area / circle area = $4r^2 / \pi r^2 = 4/\pi \approx 1.27$

So it is efficient.

24. Full Problem Walkthrough 8 — LeetCode 470: Implement Rand10 Using Rand7

24.1 Problem Summary

Given API:

```
rand7()
```

returns integer from 1 to 7 uniformly.

Implement:

```
rand10()
```

returns integer from 1 to 10 uniformly.

24.2 Key Idea — Rejection Sampling

Use two rand7 calls to generate uniform number from 1 to 49.

```
row = rand7()
```

```
col = rand7()
```

```
num = (row - 1) * 7 + col
```

num is uniform in:

```
1..49
```

Use only 1..40 because 40 is divisible by 10.

Reject 41..49.

24.3 C++ Code

```
class Solution {
public:
    int rand10() {
        while (true) {
            int row = rand7();
            int col = rand7();

            int num = (row - 1) * 7 + col;

            if (num <= 40) {
                return 1 + (num - 1) % 10;
            }
        }
    }
};
```

24.4 Why Not Use rand7() % 10?

rand7 only gives 7 outcomes, not enough for 10 uniform outcomes.

Need generate larger uniform sample space and reject excess.

25. Full Problem Walkthrough 9 — LeetCode 710: Random Pick With Blacklist

25.1 Problem Summary

Pick random integer from:

0 to $n - 1$

excluding blacklisted values.

Need efficient pick.

25.2 Key Idea

Let allowed count be:

$\text{bound} = n - \text{blacklist.size}()$

Pick random x from:

0 to $\text{bound} - 1$

If x is not blacklisted, return x .

If x is blacklisted, map it to a valid number in:

bound to $n - 1$

that is not blacklisted.

25.3 C++ Code

```
class Solution {
private:
    int bound;
    unordered_map<int, int> mapping;
    mt19937 rng;

public:
    Solution(int n, vector<int>& blacklist) {
        rng = mt19937(random_device{}());

        bound = n - blacklist.size();

        unordered_set<int> black(blacklist.begin(), blacklist.end());

        int candidate = bound;

        for (int b : blacklist) {
            if (b < bound) {
                while (black.count(candidate)) {
```

```

        candidate++;
    }

    mapping[b] = candidate;
    candidate++;
}
}
}

int pick() {
    uniform_int_distribution<int> dist(0, bound - 1);

    int x = dist(rng);

    if (mapping.count(x)) {
        return mapping[x];
    }

    return x;
}
};

```

25.4 Why This Works

There are exactly bound allowed numbers.

We generate one of bound slots uniformly.

Blacklisted slots in the lower range are remapped to allowed slots in the upper range.

Thus every allowed number has one representative.

26. Full Problem Walkthrough 10 — Randomized Collection With Duplicates

26.1 Problem Summary

Design a collection allowing duplicates.

Support:

```

insert
remove
getRandom

```

Average $O(1)$.

26.2 Difference From RandomizedSet

A value can appear multiple times.

Need map:

value -> set of indices in vector

26.3 C++ Code

```
class RandomizedCollection {
private:
    vector<int> values;
    unordered_map<int, unordered_set<int>> indices;
    mt19937 rng;

public:
    RandomizedCollection() {
        rng = mt19937(random_device{}());
    }

    bool insert(int val) {
        bool isNew = indices[val].empty();

        indices[val].insert(values.size());
        values.push_back(val);

        return isNew;
    }

    bool remove(int val) {
        if (indices[val].empty()) {
            return false;
        }

        int removeIdx = *indices[val].begin();
        indices[val].erase(removeIdx);

        int lastVal = values.back();
        int lastIdx = values.size() - 1;

        values[removeIdx] = lastVal;

        if (removeIdx != lastIdx) {
            indices[lastVal].erase(lastIdx);
            indices[lastVal].insert(removeIdx);
        }
    }
};
```

```

        values.pop_back();

    return true;
}

int getRandom() {
    uniform_int_distribution<int> dist(0, values.size() - 1);
    return values[dist(rng)];
}
};

```

26.4 Why Random Is Weighted by Frequency

Because vector contains duplicates.

If value appears k times in total size n , it occupies k indices.

So probability:

$$k / n$$

27. Full Problem Walkthrough 11 — Random Flip Matrix

27.1 Problem Summary

Given $m \times n$ binary matrix initially all 0.

`flip()` randomly picks a 0, changes it to 1, and returns position.

Each remaining 0 must be equally likely.

`reset()` resets all to 0.

27.2 Key Idea

There are total slots:

$$m * n$$

Use virtual array of remaining values.

When picking random index r in $[0, \text{remaining} - 1]$:

- actual value is `mapping[r]` if exists else r
- then map r to last remaining value
- decrease remaining

This is like Fisher-Yates without storing full array.

27.3 C++ Code

```

class Solution {
private:
    int rows;
    int cols;
    int remaining;
    unordered_map<int, int> mapping;

```

```

mt19937 rng;

public:
    Solution(int m, int n) {
        rows = m;
        cols = n;
        remaining = m * n;
        rng = mt19937(random_device{}());
    }

    vector<int> flip() {
        uniform_int_distribution<int> dist(0, remaining - 1);

        int r = dist(rng);

        int value = mapping.count(r) ? mapping[r] : r;

        remaining--;

        int lastValue = mapping.count(remaining) ? mapping[remaining] : remaining;

        mapping[r] = lastValue;

        return {value / cols, value % cols};
    }

    void reset() {
        mapping.clear();
        remaining = rows * cols;
    }
};

```

27.4 Why It Is Uniform

Each remaining unflipped cell corresponds to exactly one slot in the virtual range.
Picking uniformly from the range picks uniformly among remaining cells.

28. Expected Value Example 1 — Expected Coin Flips Until First Head

28.1 Problem

A fair coin is flipped until first head.
 What is expected number of flips?

28.2 Equation Method

Let E be expected flips.

First flip always happens.

With probability $1/2$, it is heads and stop.

With probability $1/2$, it is tails and we are back to same problem.

$$E = 1 + (1/2) * E$$

So:

$$E / 2 = 1$$

$$E = 2$$

Answer:

2

29. Expected Value Example 2 — Expected Rolls Until First 6

Same logic.

Let E be expected rolls.

Each roll succeeds with probability:

$$1 / 6$$

Fails with probability:

$$5 / 6$$

Equation:

$$E = 1 + (5/6)E$$

So:

$$E/6 = 1$$

$$E = 6$$

General rule:

If success probability is p , expected trials until first success:

$$1 / p$$

30. Expected Value Example 3 — Expected Number of Fixed Points

30.1 Problem

Randomly shuffle n distinct elements.

Expected number of positions i where:

$$\text{perm}[i] == i$$

30.2 Indicator Variables

Let $X_i = 1$ if position i is fixed.

Probability position i is fixed:

$$1 / n$$

So:

$$E[X_i] = 1/n$$

Total fixed points:

$$X = X_1 + X_2 + \dots + X_n$$

By linearity:

$$E[X] = n * 1/n = 1$$

Answer:

$$1$$

This is true for any n .

31. Expected Value Example 4 — Expected Inversions in Random Permutation

31.1 Problem

For random permutation of n distinct numbers, expected number of inversions.

An inversion is pair $i < j$ such that:

$$\text{perm}[i] > \text{perm}[j]$$

31.2 Indicator Variables

For each pair (i, j) , define $X_{ij} = 1$ if pair is inverted.

For any pair, either order is equally likely.

$$P(\text{inverted}) = 1 / 2$$

Number of pairs:

$$C(n, 2) = n(n - 1)/2$$

Expected inversions:

$$C(n, 2) * 1/2 = n(n - 1)/4$$

32. Full Problem Walkthrough 12 — Random Pick With Prefix Sum Generalization

This pattern applies beyond LeetCode 528.

32.1 Problem

Given items:

A, B, C

with weights:

5, 2, 3

Pick item according to weight.

32.2 Intervals

Prefix sums:

A: 1..5

B: 6..7

C: 8..10

Uniformly pick number in 1..10.

32.3 Code Template

```
class WeightedPicker {
private:
    vector<long long> prefix;
    long long total;
    mt19937 rng;

public:
    WeightedPicker(vector<int>& weights) {
        rng = mt19937(random_device{}());

        total = 0;

        for (int w : weights) {
            total += w;
            prefix.push_back(total);
        }
    }

    int pick() {
        uniform_int_distribution<long long> dist(1, total);
```

```

    long long x = dist(rng);

    return lower_bound(prefix.begin(), prefix.end(), x) - prefix.begin();
}
};

```

Use long long if weights can sum large.

33. Random Sampling Without Replacement

33.1 Problem

Pick k unique random elements from n .

33.2 Options

Option 1 — Shuffle

Shuffle array and take first k .

Time:

$O(n)$

Option 2 — Partial Fisher-Yates

Only shuffle first k positions.

```

for (int i = 0; i < k; i++) {
    int j = randomInt(i, n - 1);
    swap(arr[i], arr[j]);
}

```

Time:

$O(k)$

if array exists.

Option 3 — Reservoir Sampling

For stream unknown length, keep reservoir of size k .

34. Reservoir Sampling k Items

34.1 Problem

From stream of unknown length n , pick k items uniformly.

34.2 Algorithm

17. Put first k items into reservoir.
18. For item i , where i is 1-indexed:
 - choose random j from $1..i$
 - if $j \leq k$, replace $\text{reservoir}[j - 1]$

34.3 Code

```

vector<int> reservoirSample(vector<int>& stream, int k) {
    vector<int> reservoir;
    mt19937 rng(random_device{}());

    for (int i = 0; i < stream.size(); i++) {
        if (i < k) {
            reservoir.push_back(stream[i]);
        } else {
            uniform_int_distribution<int> dist(0, i);
            int j = dist(rng);

            if (j < k) {
                reservoir[j] = stream[i];
            }
        }
    }

    return reservoir;
}

```

34.4 Probability

Each item ends in reservoir with probability:

k / n

This is the k-item generalization of single-item reservoir sampling.

35. Rejection Sampling

35.1 Idea

Generate from an easy distribution.

Reject invalid samples.

Repeat until valid.

Used in:

- rand10 from rand7
- random point in circle
- random points avoiding blacklist sometimes
- generating random values under constraints

35.2 Requirement

The accepted outcomes must be symmetric/uniform.

Example:

For rand10:

Generate uniform 1..49.

Accept 1..40.

Map evenly to 1..10.

Each output has exactly 4 accepted values.

36. Common Randomized Algorithm Edge Cases

36.1 Empty Structure

getRandom() is usually guaranteed valid, but confirm.

36.2 Large Weight Sum

Use long long.

36.3 Duplicates

If duplicates should affect probability, keep duplicates in vector.

If picking unique values, use set/map.

36.4 Blacklist Large n

Cannot build array of size n if n is huge.

Use mapping trick.

36.5 Floating Precision

For geometric random point generation, small floating errors are acceptable.

36.6 Random Seed

Avoid reseeding every function call.

Seed once in constructor.

37. Common Probability / Random Mistakes

Mistake 1 — Biased Shuffle

Wrong:

```
for each i:  
    swap(i, random index from 0..n-1)
```

This is not uniform.

Correct Fisher-Yates:

```
swap(i, random index from 0..i)
```

or forward version:

```
swap(i, random index from i..n-1)
```

Mistake 2 — Uniform Radius in Circle

Choosing radius uniformly creates center bias.

Use:

```
radius * sqrt(U)
```

or rejection sampling.

Mistake 3 — Forgetting Weight Intervals

For weighted pick, random number should be based on total weight, not number of indices.

Mistake 4 — Modulo Bias

`rand() % n` can be biased.

Use uniform distribution or rejection sampling.

Mistake 5 — Removing From Vector Without Updating Map

For `RandomizedSet`, after swapping with last element, update its index.

Mistake 6 — Reservoir Sampling Wrong Probability

At k th matching item, choose with probability $1/k$, not $1/n$.

38. Interview Explanation Templates

38.1 Weighted Random Pick

I convert weights into prefix-sum intervals. Then I pick a random number from 1 to total weight and binary search which interval contains it.

38.2 Reservoir Sampling

As I scan the stream, when I have seen k matching items, I choose the current one with probability $1/k$. This guarantees every matching item has equal final probability.

38.3 Fisher-Yates Shuffle

For each position, I randomly choose one of the remaining elements and swap it into that position. This gives each permutation probability $1/n!$.

38.4 RandomizedSet

I use a vector for $O(1)$ random access and a hash map from value to index for $O(1)$ insert/delete. Delete is handled by swapping with the last element.

38.5 Rejection Sampling

I generate from a larger uniform sample space, reject extra outcomes so the remaining count is divisible by the target outcome count, then map evenly.

39. Complexity Patterns

Random Pick Index with map	$O(1)$ pick	$O(n)$
Random Pick Index reservoir	$O(n)$ pick	$O(1)$
Weighted Pick	$O(\log n)$ pick	$O(n)$
RandomizedSet	$O(1)$ avg all ops	$O(n)$
Shuffle	$O(n)$	$O(n)$ output/copy
Linked List Random Node	$O(n)$ pick	$O(1)$

Problem / Pattern	Time	Space
Random Rectangles	$O(\log r)$ pick	$O(r)$
Random Point in Circle polar	$O(1)$	$O(1)$
rand10 from rand7	expected $O(1)$	$O(1)$
Blacklist Pick	$O(1)$ pick	$O(b)$
Random Flip Matrix	$O(1)$ avg flip	$O(\text{flips})$

40. Expanded Practice Set

40.1 Must-Do Medium

19. Random Pick Index
20. Random Pick With Weight
21. Insert Delete GetRandom $O(1)$
22. Shuffle an Array
23. Linked List Random Node
24. Implement Rand10 Using Rand7

40.2 Strong Interview Practice

25. Random Point in Non-overlapping Rectangles
26. Generate Random Point in a Circle
27. Random Pick With Blacklist
28. Random Flip Matrix
29. Randomized Collection with Duplicates
30. Reservoir Sampling k Items

40.3 Probability Reasoning Practice

31. Expected coin flips until first head
32. Expected rolls until first six
33. Expected fixed points in permutation
34. Expected inversions in permutation
35. Birthday paradox preview
36. Coupon collector preview

40.4 Hard / Stretch

37. Random Pick With Weight with updates
 38. Alias method for $O(1)$ weighted sampling
 39. Randomized quicksort expected time proof
 40. Monte Carlo primality testing preview
 41. Random generator transformation problems
-

41. Suggested Study Order

Step 1 — Uniform Random Basics

Solve:

42. Shuffle an Array
43. Random Pick Index

44. Linked List Random Node

Goal:

Understand uniform selection.

Step 2 — Data Structure Randomness

Solve:

- 45. Insert Delete GetRandom $O(1)$
- 46. Randomized Collection

Goal:

Combine vector and hash map.

Step 3 — Weighted Randomness

Solve:

- 47. Random Pick With Weight
- 48. Random Point in Rectangles

Goal:

Use prefix sums as probability intervals.

Step 4 — Rejection Sampling

Solve:

- 49. Rand10 Using Rand7
- 50. Random Point in Circle

Goal:

Avoid biased random transformations.

Step 5 — Advanced Sampling

Solve:

- 51. Random Pick With Blacklist
- 52. Random Flip Matrix
- 53. Reservoir Sampling k Items

Goal:

Handle huge spaces efficiently.

Step 6 — Probability Reasoning

Practice:

- 54. expected value equations
- 55. linearity of expectation
- 56. fixed points and inversions

Goal:

Prepare for DE Shaw-style probability questions.

42. Pattern Decision Table

pick random element from array	random index
pick random target occurrence	index map / reservoir sampling
pick by weights	prefix sum + binary search
insert/delete/random $O(1)$	vector + hashmap

Problem Clue	Technique
shuffle uniformly	Fisher-Yates
linked list random node	reservoir sampling
stream random sample	reservoir sampling
random integer point in rectangles	weighted rectangle + local random
random point in circle	sqrt radius / rejection
rand10 from rand7	rejection sampling
huge range with blacklist	remap lower blacklisted values
random unflipped matrix cell	virtual Fisher-Yates mapping
expected count of events	indicator variables

43. Mini Problem 1 — Pick Random Max Index

Given array nums, pick random index among indices holding maximum value.
Use reservoir sampling.

```
int pickRandomMaxIndex(vector<int>& nums) {
    int best = INT_MIN;
    int count = 0;
    int answer = -1;

    mt19937 rng(random_device{}());

    for (int i = 0; i < nums.size(); i++) {
        if (nums[i] > best) {
            best = nums[i];
            count = 1;
            answer = i;
        } else if (nums[i] == best) {
            count++;
        }

        uniform_int_distribution<int> dist(1, count);

        if (dist(rng) == 1) {
            answer = i;
        }
    }
}
```

```
    }  
}  
  
    return answer;  
}
```

44. Mini Problem 2 — Random Boolean With Probability p/q

Return true with probability:

```
p / q  
bool randomProbability(int p, int q) {  
    static mt19937 rng(random_device{}());  
  
    uniform_int_distribution<int> dist(1, q);  
  
    return dist(rng) <= p;  
}
```

45. Mini Problem 3 — Random From Cumulative Distribution

Given cumulative probabilities scaled as integers:

[10, 30, 100]

Meaning:

index 0: 10%
index 1: 20%
index 2: 70%

Pick x in $1..100$ and `lower_bound`.

```
int pickFromCDF(vector<int>& cdf) {  
    static mt19937 rng(random_device{}());  
  
    uniform_int_distribution<int> dist(1, cdf.back());  
  
    int x = dist(rng);  
  
    return lower_bound(cdf.begin(), cdf.end(), x) - cdf.begin();  
}
```

46. Mini Problem 4 — Simulate Biased Coin From Fair Coin

Return heads with probability $3/4$.

Flip two fair coins.

Outcomes:

HH
HT
TH
TT

Return heads for any outcome except TT.

```
bool biasedCoinThreeFourth() {  
    bool a = fairCoin();  
    bool b = fairCoin();  
  
    return a || b;  
}
```

47. Common Edge Case Checklist

Before submitting randomized code, test logically:

single element
all equal weights
one huge weight
duplicate values
remove last element
remove only element
blacklist empty
blacklist contains low values
blacklist contains high values
circle radius small
rectangles with one point
linked list length one
shuffle length one

For probability, verify:

Do all valid outcomes have equal number of preimages?
Does rejection discard symmetric excess?
Is every item represented exactly once?
Are weights proportional to interval sizes?

48. Debugging Randomized Algorithms

If randomized solution fails, check:

- 57. Are you choosing from correct range inclusive/exclusive?
 - 58. Are probabilities exactly uniform?
 - 59. Are weights converted to prefix sums correctly?
 - 60. Are you using lower_bound correctly?
 - 61. Did you seed random generator only once?
 - 62. Does remove update swapped index?
 - 63. Does blacklist mapping avoid blacklisted upper values?
 - 64. Does reservoir sampling use current count?
 - 65. Does shuffle use Fisher-Yates?
 - 66. Does circle generation avoid center bias?
-

49. What to Master Before Chapter 53

Before moving to Game Theory, make sure you can:

- Explain probability as favorable / total
 - Explain expected value
 - Use linearity of expectation
 - Solve Random Pick Index
 - Explain reservoir sampling proof
 - Solve Random Pick With Weight
 - Use prefix sums for weighted random
 - Solve RandomizedSet
 - Implement Fisher-Yates shuffle
 - Explain why naive shuffle is biased
 - Solve rand10 from rand7
 - Explain rejection sampling
 - Solve Random Point in Circle
 - Solve Random Pick With Blacklist
 - Recognize Monte Carlo vs Las Vegas
-

50. Final Cheat Sheet

Uniform Random Integer

```
uniform_int_distribution<int> dist(l, r);  
int x = dist(rng);
```

Weighted Pick

```
x = random 1..total;  
idx = lower_bound(prefix.begin(), prefix.end(), x) - prefix.begin();
```

Reservoir Sampling One Item

```
count++;  
  
if (randomInt(1, count) == 1) {  
    answer = current;  
}
```

Fisher-Yates Shuffle

```
for (int i = n - 1; i > 0; i--) {  
    int j = randomInt(0, i);  
    swap(a[i], a[j]);  
}
```

RandomizedSet Remove

```
idx = index[val];  
last = values.back();  
  
values[idx] = last;  
index[last] = idx;  
  
values.pop_back();  
index.erase(val);
```

Rand10 From Rand7

```
num = (rand7() - 1) * 7 + rand7();  
  
if (num <= 40) {  
    return 1 + (num - 1) % 10;  
}
```

Circle Point

```
r = radius * sqrt(U);  
theta = 2 * pi * V;
```

Expected Geometric Trials

$$E = 1 / p$$

Linearity of Expectation

$$E[X_1 + X_2 + \dots + X_n] = E[X_1] + E[X_2] + \dots + E[X_n]$$

51. Core Takeaway

Randomized algorithms are about preserving the exact intended probability distribution.

The most important patterns are:

- prefix sums for weighted random
- reservoir sampling for streams
- Fisher-Yates for uniform shuffle
- vector + hashmap for $O(1)$ random set
- rejection sampling for converting random generators
- indicator variables for expected value

Always ask:

Are all valid outcomes equally likely?

If weighted, are probabilities proportional to weights?

If rejecting, is the accepted sample space evenly mappable?

For probability reasoning, remember:

Expected value is often easier than full distribution.

Linearity of expectation is your best friend.